RV32 SoC SDK API Reference

RV32 SoC Project

2026-02-25

Table of Contents

1. Abstract . . . 1
2. Introduction . . . 2
   2.1. Document Scope . . . 2
   2.2. SDK Architecture . . . 2
   2.3. Header Organization . . . 2
3. System Memory Map . . . 4
   3.1. Address Space Overview . . . 4
   3.2. Memory-Mapped Register Access . . . 4
4. UART Driver API . . . 5
   4.1. Overview . . . 5
   4.2. Register Map . . . 5
   4.3. Status Bits . . . 5
   4.4. API Functions . . . 5
      4.4.1. Initialization . . . 5
      4.4.2. Status Queries . . . 6
      4.4.3. Polling Transmit . . . 6
      4.4.4. Polling Receive . . . 6
      4.4.5. Interrupt Control . . . 7
      4.4.6. Loopback Mode . . . 7
5. SPI Driver API . . . 8
   5.1. Overview . . . 8
   5.2. Register Map . . . 8
   5.3. SPI Modes . . . 8
   5.4. API Functions . . . 9
      5.4.1. Initialization . . . 9
      5.4.2. Chip Select Control . . . 9
      5.4.3. Data Transfer . . . 9
      5.4.4. Status and Control . . . 10
      5.4.5. Interrupt Control . . . 10
      5.4.6. Loopback Mode . . . 10
6. I2C Driver API . . . 11
   6.1. Overview . . . 11
   6.2. Register Map . . . 11
   6.3. API Functions . . . 11
      6.3.1. Initialization . . . 11
      6.3.2. Low-Level Operations . . . 12
      6.3.3. High-Level Transfers . . . 12
      6.3.4. Status and Control . . . 13

6.3.5. Interrupt Control . . . 13
7. DMA Controller API . . . 14
7.1. Overview . . . 14
7.2. Register Map . . . 14
7.2.1. Per-Channel Registers . . . 14
7.2.2. Global Registers . . . 15
7.3. Control Register Bits . . . 15
7.4. Status Register Bits . . . 15
7.5. Transfer Modes . . . 15
7.5.1. 1D Transfer Mode . . . 15
7.5.2. 2D Strided Transfer Mode . . . 16
7.5.3. 3D Planar Transfer Mode . . . 16
7.6. Performance Monitoring . . . 16
8. Platform-Level Interrupt Controller (PLIC) . . . 18
8.1. Overview . . . 18
8.2. Interrupt Sources . . . 18
8.3. API Functions . . . 18
8.3.1. Source Control . . . 18
8.3.2. Interrupt Handling . . . 18
8.3.3. Initialization Helper . . . 19
9. Core-Local Interrupt Controller (CLINT) . . . 20
9.1. Overview . . . 20
9.2. Features . . . 20
9.3. API Functions . . . 20
9.3.1. Timer Operations . . . 20
9.3.2. Software Interrupts . . . 21
10. Interrupt Management API . . . 22
10.1. Overview . . . 22
10.2. Interrupt Source Bits (mie/mip) . . . 22
10.3. Interrupt Cause Codes (mcause) . . . 22
10.4. Exception Cause Codes . . . 22
10.5. API Functions . . . 22
10.5.1. Global Interrupt Control . . . 22
10.5.2. Per-Source Control . . . 23
11. Control and Status Registers (CSR) . . . 24
11.1. Overview . . . 24
11.2. Machine Information Registers . . . 24
11.3. Machine Trap Setup . . . 24
11.4. Machine Trap Handling . . . 24
11.5. Performance Counters . . . 25
12. Magic Device (Simulation/Test) . . . 26

12.1. Overview . . . 26
12.2. Registers . . . 26
12.3. Exit Code Encoding . . . 26
12.4. API Functions . . . 26
13. Complete Examples . . . 28
13.1. Example 1: UART Echo with Interrupts . . . 28
13.2. Example 2: SPI Flash Read . . . 29
13.3. Example 3: I2C EEPROM Access . . . 29
13.4. Example 4: DMA Memory Copy with Performance Monitoring . . . 31
14. Build System Integration . . . 33
14.1. Makefile Configuration . . . 33
14.2. Linker Script . . . 33
14.3. Startup Code . . . 33
14.4. Common Runtime . . . 33
15. Appendix A: Register Quick Reference . . . 34
15.1. UART Registers . . . 34
15.2. SPI Registers . . . 34
15.3. I2C Registers . . . 34
15.4. DMA Channel Registers . . . 35
16. Appendix B: Interrupt Vector Table . . . 36
17. Appendix C: Performance Considerations . . . 37
17.1. DMA vs CPU Copy . . . 37
17.2. Cache Coherency . . . 37
17.3. Interrupt Latency . . . 37
18. Appendix D: Debugging Tips . . . 38
18.1. Enable Instruction Trace . . . 38
18.2. Generate Waveforms . . . 38
18.3. Common Issues . . . 38
Document Information . . . 39

Chapter 1. Abstract

This document provides a comprehensive reference for the RV32 SoC Software Development Kit (SDK). It describes the API functions, register definitions, and programming model for all peripherals and system components in the RV32 RISC-V System-on-Chip.

The SDK provides a header-only C library optimized for bare-metal embedded applications, with zero overhead when functions are not used.

Chapter 2. Introduction

2.1. Document Scope

This API reference covers:

- Memory-mapped I/O (MMIO) peripheral drivers
- Interrupt controller configuration (PLIC, CLINT)
- Control and Status Register (CSR) access
- System initialization and exception handling
- Debug and simulation utilities

2.2. SDK Architecture

The RV32 SDK follows these design principles:

Header-Only Design

All driver functions are implemented as `static inline` functions, eliminating the need to link against a separate library. Unused functions contribute zero code size.

Zero Overhead Abstraction

Direct register access with no additional memory or CPU overhead compared to raw MMIO operations.

Type Safety

All register addresses and bit fields are defined as compile-time constants with proper types.

Portability

Works with both RTL simulation and software simulators (rv32sim, Spike).

2.3. Header Organization

Header File	Peripheral/Module	Description
`rv_platform.h`	Platform definitions	Base addresses, register offsets, bit-field constants for all peripherals
`rv_uart.h`	UART	Serial communication driver (polling and interrupt modes)
`rv_spi.h`	SPI	SPI master controller driver
`rv_i2c.h`	I2C	I2C master controller driver
`rv_plic.h`	PLIC	Platform-Level Interrupt Controller configuration

Header File	Peripheral/Module	Description
`rv_clint.h`	CLINT	Core-Local Interrupt Controller (timer and software interrupts)
`rv_irq.h`	Interrupt Management	Global and per-source interrupt control, handler registration
`csr.h`	CSR Access	Control and Status Register read/write functions

Chapter 3. System Memory Map

3.1. Address Space Overview

Region	Base Address	Size	Description
Boot ROM	`0x0000_0000`	64 KB	Read-only boot code (synthesized firmware)
Main RAM	`0x8000_0000`	2 MB	Program and data memory (SRAM)
CLINT	`0x0200_0000`	64 KB	Core-Local Interrupt Controller
PLIC	`0x0C00_0000`	64 MB	Platform-Level Interrupt Controller
UART	`0x2000_0000`	64 KB	Serial communication peripheral
I2C	`0x2001_0000`	64 KB	I2C master controller
SPI	`0x2002_0000`	64 KB	SPI master controller
DMA	`0x2003_0000`	4 KB	Direct Memory Access controller (8 channels)
Magic Device	`0xFFFF_0000`	64 KB	Simulation/testbench console and exit control

3.2. Memory-Mapped Register Access

All peripherals use 32-bit aligned registers. The SDK provides the `RV_REG32` macro for safe MMIO access:

```
#define RV_REG32(base, off) \
    (*(volatile uint32_t *)((uintptr_t)(base) + (uintptr_t)(off)))
```

Example usage:

```
uint32_t status = RV_REG32(RV_UART_BASE, RV_UART_STATUS_OFF);
RV_REG32(RV_UART_BASE, RV_UART_DATA_OFF) = 0x41; // Write 'A'
```

Chapter 4. UART Driver API

4.1. Overview

The UART peripheral (`axi_uart.sv`) provides full-duplex serial communication with:

- Configurable baud rate
- 8-bit data, no parity, 1 stop bit (8N1 format)
- TX and RX FIFOs (16 entries each)
- Interrupt generation on RX data available and TX FIFO empty
- Internal loopback mode for testing

Base Address

`0x2000_0000` (RV_UART_BASE)

PLIC Source ID

1 (RV_PLIC_SRC_UART)

4.2. Register Map

Offset	Name	Access	Description
`0x00`	DATA	R/W	TX (write) / RX (read) data register
`0x04`	STATUS	RO	Status flags (TX busy, RX ready, FIFO full)
`0x08`	IE	R/W	Interrupt Enable register
`0x0C`	IS	R/W1C	Interrupt Status register (write-1-to-clear)
`0x10`	LEVEL	R/W	Baud rate divisor
`0x14`	CTRL	R/W	Control register (loopback enable)

4.3. Status Bits

Bit	Name	Description
0	TX_BUSY / TX_FULL	TX FIFO full – cannot write
2	RX_READY	RX FIFO not empty – data available
3	RX_FULL	RX FIFO full

4.4. API Functions

4.4.1. Initialization

```
void rv_uart_init(uint32_t baud_div);
```

Initialize the UART peripheral with the specified baud rate divisor.

Parameters:

- `baud_div`: Baud rate divisor = `(sys_clk_hz / (baud_rate * 8)) - 1`

Example:

```
// Configure for 115200 baud at 100 MHz system clock
// baud_div = (100000000 / (115200 * 8)) - 1 = 107
rv_uart_init(107);
```

4.4.2. Status Queries

```
int rv_uart_tx_busy(void);    // Returns 1 if TX FIFO full
int rv_uart_rx_ready(void);   // Returns 1 if RX data available
```

4.4.3. Polling Transmit

```
void rv_uart_putc(char c);
void rv_uart_puts(const char *s);
void rv_uart_write(const uint8_t *buf, uint32_t len);
```

Description:

- `rv_uart_putc()`: Block until TX FIFO has space, then transmit one character
- `rv_uart_puts()`: Transmit a null-terminated string
- `rv_uart_write()`: Transmit len bytes from buffer

Example:

```
rv_uart_puts("Hello, World!\n");
```

4.4.4. Polling Receive

```
int rv_uart_getc(void);              // Non-blocking, returns -1 if empty
uint8_t rv_uart_getc_blocking(void); // Blocks until data available
```

Example:

```
int c;
while ((c = rv_uart_getc()) != -1) {
    rv_uart_putc((char)c);  // Echo received character
}
```

4.4.5. Interrupt Control

```
void rv_uart_irq_enable(uint32_t mask);
void rv_uart_irq_disable(uint32_t mask);
uint32_t rv_uart_irq_status(void);
```

Interrupt Sources:

- `RV_UART_IE_RX_READY` (bit 0): RX FIFO not empty
- `RV_UART_IE_TX_EMPTY` (bit 1): TX FIFO drained (empty)

Example:

```
// Enable RX interrupt
rv_uart_irq_enable(RV_UART_IE_RX_READY);

// In interrupt handler:
uint32_t status = rv_uart_irq_status();
if (status & RV_UART_IE_RX_READY) {
    char c = rv_uart_getc();
    // Process received character
}
```

4.4.6. Loopback Mode

```
void rv_uart_loopback_enable(void);
void rv_uart_loopback_disable(void);
```

Enable internal TX→RX loopback for hardware testing. Transmitted bytes are fed back to the RX path without leaving the chip.

Chapter 5. SPI Driver API

5.1. Overview

The SPI peripheral (`axi_spi.sv`) provides a master-mode SPI controller with:

- Configurable clock frequency
- All four SPI modes (CPOL, CPHA)
- Up to 4 independent chip-select lines
- TX and RX FIFOs (16 entries each)
- Full-duplex operation
- Internal loopback mode

Base Address

`0x2002_0000` (RV_SPI_BASE)

PLIC Source ID

2 (RV_PLIC_SRC_SPI)

5.2. Register Map

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register (enable, CPOL, CPHA, CS, loopback)
`0x04`	DIV	R/W	Clock divisor
`0x08`	TX	WO	TX FIFO push
`0x0C`	RX	RO	RX FIFO pop
`0x10`	STATUS	RO	Status flags
`0x14`	IE	R/W	Interrupt Enable
`0x18`	IS	R/W1C	Interrupt Status

5.3. SPI Modes

Mode	CPOL	CPHA	Description
MODE0	0	0	Clock idle low, sample on rising edge
MODE1	0	1	Clock idle low, sample on falling edge
MODE2	1	0	Clock idle high, sample on falling edge
MODE3	1	1	Clock idle high, sample on rising edge

5.4. API Functions

5.4.1. Initialization

```
void rv_spi_init(uint32_t clk_div, uint32_t mode);
```

Parameters:

- `clk_div`: SCK frequency = `sys_clk / (2 * (clk_div + 1))`
- `mode`: `RV_SPI_MODE0`, `MODE1`, `MODE2`, or `MODE3`

Example:

```
// Configure for 1 MHz SPI clock at 100 MHz system clock, mode 0
// clk_div = (100000000 / (2 * 1000000)) - 1 = 49
rv_spi_init(49, RV_SPI_MODE0);
```

5.4.2. Chip Select Control

```
void rv_spi_cs_select(uint32_t n);   // Assert CS[n] (0-3)
void rv_spi_cs_deselect(void);       // Deassert all CS lines
```

Example:

```
rv_spi_cs_select(0);        // Assert CS0
// Perform transfer
rv_spi_cs_deselect();       // Release CS0
```

5.4.3. Data Transfer

```
uint8_t rv_spi_transfer(uint8_t data);
void rv_spi_transfer_buf(const uint8_t *tx_buf, uint8_t *rx_buf, uint32_t len);
void rv_spi_read_buf(uint8_t *rx_buf, uint32_t len);
```

Description:

- `rv_spi_transfer()`: Full-duplex transfer of one byte
- `rv_spi_transfer_buf()`: Transfer multiple bytes (tx_buf or rx_buf may be NULL)
- `rv_spi_read_buf()`: Read len bytes by clocking out 0xFF

Example:

```
uint8_t tx_data[] = {0x03, 0x00, 0x00, 0x00}; // Read command + address
uint8_t rx_data[256];

rv_spi_cs_select(0);
rv_spi_transfer_buf(tx_data, NULL, 4);  // Send command
rv_spi_read_buf(rx_data, 256);          // Read data
rv_spi_cs_deselect();
```

5.4.4. Status and Control

```
int rv_spi_busy(void);        // Transfer in progress
int rv_spi_tx_ready(void);    // Can accept TX data
int rv_spi_rx_valid(void);    // RX data available
void rv_spi_wait_ready(void); // Block until idle
```

5.4.5. Interrupt Control

```
void rv_spi_irq_enable(uint32_t mask);
void rv_spi_irq_disable(uint32_t mask);
uint32_t rv_spi_irq_status(void);
```

Interrupt Sources:

- `RV_SPI_IE_RX_READY` (bit 0): RX FIFO not empty
- `RV_SPI_IE_TX_EMPTY` (bit 1): TX FIFO empty

5.4.6. Loopback Mode

```
void rv_spi_loopback_enable(void);
void rv_spi_loopback_disable(void);
```

Chapter 6. I2C Driver API

6.1. Overview

The I2C peripheral (`axi_i2c.sv`) provides a master-mode I2C controller with:

- Configurable clock frequency (100 kHz / 400 kHz / custom)
- 7-bit addressing
- START/STOP condition generation
- ACK/NACK detection
- Interrupt support

Base Address

`0x2001_0000` (RV_I2C_BASE)

PLIC Source ID

3 (RV_PLIC_SRC_I2C)

6.2. Register Map

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register (enable, START, STOP, READ, NACK)
`0x04`	DIV	R/W	Clock divisor
`0x08`	TX	WO	Transmit data register
`0x0C`	RX	RO	Receive data register
`0x10`	STATUS	RO	Status flags (busy, ACK received, RX valid)
`0x14`	IE	R/W	Interrupt Enable
`0x18`	IS	R/W1C	Interrupt Status

6.3. API Functions

6.3.1. Initialization

```
void rv_i2c_init(uint32_t clk_div);
```

Parameters:

- `clk_div`: SCL period = `sys_clk / (4 * (clk_div + 1))`

Example:

```
// Configure for 100 kHz at 100 MHz system clock
// clk_div = (100000000 / (4 * 100000)) - 1 = 249
rv_i2c_init(249);

// Configure for 400 kHz at 100 MHz system clock
// clk_div = (100000000 / (4 * 400000)) - 1 = 62
rv_i2c_init(62);
```

6.3.2. Low-Level Operations

```
void rv_i2c_start(void);                    // Issue START condition
void rv_i2c_stop(void);                     // Issue STOP condition
int rv_i2c_write_byte(uint8_t data);        // Write byte, return 0 if ACK
uint8_t rv_i2c_read_byte(int send_ack);     // Read byte, send ACK if 1
```

6.3.3. High-Level Transfers

```
int rv_i2c_master_write(uint8_t addr, const uint8_t *buf, uint32_t len);
int rv_i2c_master_read(uint8_t addr, uint8_t *buf, uint32_t len);
```

Description:

Complete write or read transactions including START, address, data, and STOP.

Parameters:

- `addr`: 7-bit I2C device address
- `buf`: Data buffer
- `len`: Number of bytes to transfer

Return Values:

- `0`: Success
- `-1`: Device did not ACK the address
- `-2`: Device did not ACK a data byte

Example:

```
// Write to EEPROM at address 0x50
uint8_t eeprom_addr = 0x50;
uint8_t write_data[] = {0x00, 0x10, 0xAA, 0xBB}; // Addr MSB, LSB, data

if (rv_i2c_master_write(eeprom_addr, write_data, 4) < 0) {
    // Handle error
}
```

```
// Read from EEPROM
uint8_t read_buf[2];
if (rv_i2c_master_read(eeprom_addr, read_buf, 2) < 0) {
    // Handle error
}
```

6.3.4. Status and Control

```
int rv_i2c_busy(void);           // Transfer in progress
int rv_i2c_ack_received(void);   // Slave ACKed last byte
int rv_i2c_rx_valid(void);       // RX data available
void rv_i2c_wait_ready(void);    // Block until idle
```

6.3.5. Interrupt Control

```
void rv_i2c_irq_enable(uint32_t mask);
void rv_i2c_irq_disable(uint32_t mask);
uint32_t rv_i2c_irq_status(void);
```

Interrupt Sources:

- `RV_I2C_IE_RX_READY` (bit 0): RX FIFO not empty
- `RV_I2C_IE_TX_EMPTY` (bit 1): TX FIFO empty
- `RV_I2C_IE_STOP_DONE` (bit 2): STOP condition completed

Chapter 7. DMA Controller API

7.1. Overview

The DMA controller (`axi_dma.sv`) provides high-performance data transfers with:

- 8 independent channels
- Multiple transfer modes: 1D, 2D strided, 3D planar, scatter-gather
- Configurable source/destination address increment
- Interrupt generation on completion or error
- Performance counters for throughput monitoring

Base Address

`0x2003_0000` (RV_DMA_BASE)

PLIC Source ID

4 (RV_PLIC_SRC_DMA)

7.2. Register Map

7.2.1. Per-Channel Registers

Each channel has registers at `RV_DMA_BASE + channel * 0x40`:

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register
`0x04`	STAT	R/W1C	Status register
`0x08`	SRC	R/W	Source address
`0x0C`	DST	R/W	Destination address
`0x10`	XFER	R/W	Transfer size (bytes)
`0x14`	SSTRIDE	R/W	Source stride (2D mode)
`0x18`	DSTRIDE	R/W	Destination stride (2D mode)
`0x1C`	ROWCNT	R/W	Row count (2D/3D mode)
`0x20`	SPSTRIDE	R/W	Source plane stride (3D mode)
`0x24`	DPSTRIDE	R/W	Destination plane stride (3D mode)
`0x28`	PLANECNT	R/W	Plane count (3D mode)
`0x2C`	SGADDR	R/W	Scatter-gather descriptor address
`0x30`	SGCNT	R/W	Scatter-gather descriptor count

7.2.2. Global Registers

Offset	Name	Access	Description
`0xF00`	IRQ_STAT	R/W1C	Channel done/error flags (one bit per channel)
`0xF04`	IRQ_EN	R/W	Global interrupt enable per channel
`0xF08`	ID	RO	Identification register (0xD4A00100)
`0xF10`	PERF_CTRL	R/W	Performance counter control
`0xF14`	PERF_CYCLES	R/W	Cycles elapsed
`0xF18`	PERF_RD_BYTES	R/W	Total bytes read
`0xF1C`	PERF_WR_BYTES	R/W	Total bytes written

7.3. Control Register Bits

Bit(s)	Name	Description
0	EN	Enable channel
1	START	Arm transfer (auto-clears)
2	STOP	Abort transfer (auto-clears)
4:3	MODE	`00`=1D, `01`=2D, `10`=3D, `11`=Scatter-Gather
5	SRC_INC	Increment source address
6	DST_INC	Increment destination address
7	IE	Interrupt enable for this channel

7.4. Status Register Bits

Bit	Name	Description
0	BUSY	Transfer in progress (read-only)
1	DONE	Transfer complete (write-1-to-clear)
2	ERR	AXI error occurred (write-1-to-clear)

7.5. Transfer Modes

7.5.1. 1D Transfer Mode

Simple linear memory copy:

```
// Copy 1024 bytes from src to dst
RV_DMA_CH_REG(0, RV_DMA_CH_SRC_OFF) = src_addr;
```

```
RV_DMA_CH_REG(0, RV_DMA_CH_DST_OFF) = dst_addr;
RV_DMA_CH_REG(0, RV_DMA_CH_XFER_OFF) = 1024;
RV_DMA_CH_REG(0, RV_DMA_CH_CTRL_OFF) =
    RV_DMA_CTRL_EN | RV_DMA_CTRL_MODE_1D |
    RV_DMA_CTRL_SRC_INC | RV_DMA_CTRL_DST_INC | RV_DMA_CTRL_START;

// Wait for completion
while (RV_DMA_CH_REG(0, RV_DMA_CH_STAT_OFF) & RV_DMA_STAT_BUSY) {}
```

7.5.2. 2D Strided Transfer Mode

Copy rectangular regions with different pitches:

```
// Copy 640x480 image with padding
// Source: 640 bytes/row, stride 1024
// Dest:   640 bytes/row, stride 800
RV_DMA_CH_REG(0, RV_DMA_CH_SRC_OFF) = src_addr;
RV_DMA_CH_REG(0, RV_DMA_CH_DST_OFF) = dst_addr;
RV_DMA_CH_REG(0, RV_DMA_CH_XFER_OFF) = 640;        // Bytes per row
RV_DMA_CH_REG(0, RV_DMA_CH_SSTRIDE_OFF) = 1024;    // Source row stride
RV_DMA_CH_REG(0, RV_DMA_CH_DSTRIDE_OFF) = 800;     // Dest row stride
RV_DMA_CH_REG(0, RV_DMA_CH_ROWCNT_OFF) = 480;      // Number of rows
RV_DMA_CH_REG(0, RV_DMA_CH_CTRL_OFF) =
    RV_DMA_CTRL_EN | RV_DMA_CTRL_MODE_2D |
    RV_DMA_CTRL_SRC_INC | RV_DMA_CTRL_DST_INC | RV_DMA_CTRL_START;
```

7.5.3. 3D Planar Transfer Mode

Copy 3D data structures (e.g., video frames):

```
// Copy video buffer: 640x480x3 (RGB planes)
RV_DMA_CH_REG(0, RV_DMA_CH_SRC_OFF) = src_addr;
RV_DMA_CH_REG(0, RV_DMA_CH_DST_OFF) = dst_addr;
RV_DMA_CH_REG(0, RV_DMA_CH_XFER_OFF) = 640;           // Bytes per row
RV_DMA_CH_REG(0, RV_DMA_CH_SSTRIDE_OFF) = 640;        // Source row stride
RV_DMA_CH_REG(0, RV_DMA_CH_DSTRIDE_OFF) = 640;        // Dest row stride
RV_DMA_CH_REG(0, RV_DMA_CH_ROWCNT_OFF) = 480;         // Rows per plane
RV_DMA_CH_REG(0, RV_DMA_CH_SPSTRIDE_OFF) = 307200;    // Source plane stride
RV_DMA_CH_REG(0, RV_DMA_CH_DPSTRIDE_OFF) = 307200;    // Dest plane stride
RV_DMA_CH_REG(0, RV_DMA_CH_PLANECNT_OFF) = 3;         // Number of planes
RV_DMA_CH_REG(0, RV_DMA_CH_CTRL_OFF) =
    RV_DMA_CTRL_EN | RV_DMA_CTRL_MODE_3D |
    RV_DMA_CTRL_SRC_INC | RV_DMA_CTRL_DST_INC | RV_DMA_CTRL_START;
```

7.6. Performance Monitoring

```
// Enable performance counters
RV_DMA_GLB_REG(RV_DMA_PERF_CTRL_OFF) = 0x3; // Reset + enable

// Start DMA transfer
// ...

// Wait for completion
// ...

// Read statistics
uint32_t cycles = RV_DMA_GLB_REG(RV_DMA_PERF_CYCLES_OFF);
uint32_t rd_bytes = RV_DMA_GLB_REG(RV_DMA_PERF_RD_BYTES_OFF);
uint32_t wr_bytes = RV_DMA_GLB_REG(RV_DMA_PERF_WR_BYTES_OFF);

// Calculate throughput
float mbps = (float)(rd_bytes + wr_bytes) / cycles * sys_clk_hz / 1e6;
```

Chapter 8. Platform-Level Interrupt Controller (PLIC)

8.1. Overview

The PLIC (`axi_plic.sv`) manages external interrupt sources from peripherals.

Base Address

`0x0C00_0000` (RV_PLIC_BASE)

8.2. Interrupt Sources

ID	Source	Description
1	UART	Serial port RX ready or TX empty
2	SPI	SPI RX ready or TX empty
3	I2C	I2C RX ready, TX empty, or STOP done
4	DMA	DMA channel done or error

8.3. API Functions

8.3.1. Source Control

```
void rv_plic_set_priority(uint32_t src, uint32_t priority);
void rv_plic_enable_source(uint32_t src);
void rv_plic_disable_source(uint32_t src);
int rv_plic_is_pending(uint32_t src);
void rv_plic_set_threshold(uint32_t threshold);
```

Parameters:

- `src`: Source ID (1-31)
- `priority`: Priority level (0-7, where 0 = disabled, 7 = highest)
- `threshold`: Only sources with priority > threshold are forwarded

8.3.2. Interrupt Handling

```
uint32_t rv_plic_claim(void);
void rv_plic_complete(uint32_t src);
```

Description:

- `rv_plic_claim()`: Read the highest-priority pending interrupt source (returns 0 if none)
- `rv_plic_complete()`: Signal completion of interrupt handling

8.3.3. Initialization Helper

```
void rv_plic_init_source(uint32_t src, uint32_t priority);
```

Convenience function that sets priority, enables source, sets threshold to 0, and enables external interrupts in `mie`.

Example:

```
// Initialize PLIC for UART interrupts
rv_plic_init_source(RV_PLIC_SRC_UART, 5);  // Priority 5

// Enable UART RX interrupt
rv_uart_irq_enable(RV_UART_IE_RX_READY);

// Enable global interrupts
rv_irq_enable();

// Interrupt handler
void external_irq_handler(void) {
    uint32_t src = rv_plic_claim();

    if (src == RV_PLIC_SRC_UART) {
        uint32_t status = rv_uart_irq_status();
        if (status & RV_UART_IE_RX_READY) {
            char c = rv_uart_getc();
            // Process character
        }
    }

    rv_plic_complete(src);
}
```

Chapter 9. Core-Local Interrupt Controller (CLINT)

9.1. Overview

The CLINT (`axi_clint.sv`) provides timer and software interrupts.

Base Address

`0x0200_0000` (RV_CLINT_BASE)

9.2. Features

- 64-bit free-running timer (`mtime`)
- 64-bit timer compare register (`mtimecmp`)
- Machine software interrupt (MSIP)

9.3. API Functions

9.3.1. Timer Operations

```
uint64_t rv_clint_mtime(void);
void rv_clint_set_mtimecmp(uint64_t cmp);
void rv_clint_timer_disable(void);
void rv_clint_timer_set_rel(uint64_t ticks);
void rv_clint_timer_irq_enable(void);
void rv_clint_timer_irq_disable(void);
```

Description:

- `rv_clint_mtime()`: Read current timer value (safe against high-word rollover)
- `rv_clint_set_mtimecmp()`: Set compare value for timer interrupt
- `rv_clint_timer_set_rel()`: Schedule interrupt N ticks from now

Example:

```
// Schedule timer interrupt in 1000000 cycles (~10ms at 100 MHz)
rv_clint_timer_set_rel(1000000);
rv_clint_timer_irq_enable();
rv_irq_enable();

// Timer interrupt handler
void timer_irq_handler(void) {
    // Handle timer interrupt
```

```
    // Re-arm for next interrupt
    rv_clint_timer_set_rel(1000000);
}
```

9.3.2. Software Interrupts

```
void rv_clint_msip_set(void);
void rv_clint_msip_clear(void);
void rv_clint_msip_irq_enable(void);
void rv_clint_msip_irq_disable(void);
```

Software interrupts can be used for inter-processor communication (IPI) in multi-core systems or for self-interrupts.

Chapter 10. Interrupt Management API

10.1. Overview

The `rv_irq.h` module provides high-level interrupt management functions.

10.2. Interrupt Source Bits (mie/mip)

Constant	Bit	Description
`RV_IRQ_MSIE`	3	Machine software interrupt enable
`RV_IRQ_MTIE`	7	Machine timer interrupt enable
`RV_IRQ_MEIE`	11	Machine external interrupt enable (PLIC)

10.3. Interrupt Cause Codes (mcause)

Constant	Code	Description
`RV_CAUSE_MSI`	3	Machine software interrupt
`RV_CAUSE_MTI`	7	Machine timer interrupt
`RV_CAUSE_MEI`	11	Machine external interrupt

10.4. Exception Cause Codes

Constant	Code	Description
`RV_EXC_INSN_MISALIGN`	0	Instruction address misaligned
`RV_EXC_INSN_FAULT`	1	Instruction access fault
`RV_EXC_ILLEGAL_INSN`	2	Illegal instruction
`RV_EXC_BREAKPOINT`	3	Breakpoint (EBREAK)
`RV_EXC_LOAD_MISALIGN`	4	Load address misaligned
`RV_EXC_LOAD_FAULT`	5	Load access fault
`RV_EXC_STORE_MISALIGN`	6	Store address misaligned
`RV_EXC_STORE_FAULT`	7	Store access fault
`RV_EXC_ECALL_M`	11	Environment call from M-mode

10.5. API Functions

10.5.1. Global Interrupt Control

```
void rv_irq_enable(void);
```

```
uint32_t rv_irq_disable(void);
void rv_irq_restore(uint32_t saved_mstatus);
```

Example:

```
// Critical section
uint32_t mstatus = rv_irq_disable();
// ... critical code ...
rv_irq_restore(mstatus);
```

10.5.2. Per-Source Control

```
void rv_irq_source_enable(uint32_t mask);
void rv_irq_source_disable(uint32_t mask);
int rv_irq_source_is_enabled(uint32_t mask);
```

Example:

```
// Enable timer and external interrupts
rv_irq_source_enable(RV_IRQ_MTIE | RV_IRQ_MEIE);

// Disable timer interrupt
rv_irq_source_disable(RV_IRQ_MTIE);
```

Chapter 11. Control and Status Registers (CSR)

11.1. Overview

The `csr.h` module provides inline functions for reading and writing RISC-V control and status registers.

11.2. Machine Information Registers

```
uint32_t read_csr_mvendorid(void);  // Vendor ID
uint32_t read_csr_marchid(void);    // Architecture ID
uint32_t read_csr_mimpid(void);     // Implementation ID
uint32_t read_csr_mhartid(void);    // Hardware thread ID
```

11.3. Machine Trap Setup

```
uint32_t read_csr_mstatus(void);
void write_csr_mstatus(uint32_t val);

uint32_t read_csr_misa(void);       // ISA and extensions

uint32_t read_csr_mie(void);
void write_csr_mie(uint32_t val);

uint32_t read_csr_mtvec(void);
void write_csr_mtvec(uint32_t val);
```

11.4. Machine Trap Handling

```
uint32_t read_csr_mscratch(void);
void write_csr_mscratch(uint32_t val);

uint32_t read_csr_mepc(void);       // Exception program counter
void write_csr_mepc(uint32_t val);

uint32_t read_csr_mcause(void);     // Exception cause
void write_csr_mcause(uint32_t val);

uint32_t read_csr_mtval(void);      // Trap value (faulting address)
void write_csr_mtval(uint32_t val);

uint32_t read_csr_mip(void);        // Interrupt pending
```

```
void write_csr_mip(uint32_t val);
```

11.5. Performance Counters

```
uint64_t read_csr_mcycle(void);     // Cycle counter
uint64_t read_csr_minstret(void);   // Instructions retired counter
```

Example:

```
// Measure execution time
uint64_t start = read_csr_mcycle();
// ... code to measure ...
uint64_t end = read_csr_mcycle();
uint64_t cycles = end - start;

// Measure instruction count
uint64_t start_insn = read_csr_minstret();
// ... code to measure ...
uint64_t end_insn = read_csr_minstret();
uint64_t instructions = end_insn - start_insn;

float cpi = (float)cycles / instructions;
```

Chapter 12. Magic Device (Simulation/Test)

12.1. Overview

The Magic Device provides a simple interface for firmware to communicate with simulation environments without requiring a full UART implementation.

Base Address

`0xFFFF_0000` (RV_MAGIC_BASE)

> This peripheral is only available in simulation (RTL testbench, rv32sim, Spike plugins). It will not be synthesized for production FPGA/ASIC.

12.2. Registers

Offset	Name	Description
`0xFFF0`	EXIT	Write exit code to signal simulation end
`0xFFF4`	CONSOLE	Write character (low byte) to simulation console

12.3. Exit Code Encoding

The EXIT register uses HTIF tohost encoding:

- Success (code 0): Write `1`
- Failure (code N): Write `(N << 1) \| 1`

12.4. API Functions

```
void rv_magic_putc(char c);
void rv_magic_exit(int code);
```

Description:

- `rv_magic_putc()`: Output one character to simulation console
- `rv_magic_exit()`: Signal program exit and halt (infinite loop)

Example:

```
#include "rv_platform.h"

int main(void) {
    const char *msg = "Hello from RV32!\n";
```

```
    // Output string
    for (const char *p = msg; *p; p++) {
        rv_magic_putc(*p);
    }

    // Signal successful completion
    rv_magic_exit(0);  // Simulation will stop here

    return 0;
}
```

Chapter 13. Complete Examples

13.1. Example 1: UART Echo with Interrupts

```c
#include "rv_platform.h"
#include "rv_uart.h"
#include "rv_plic.h"
#include "rv_irq.h"

#define UART_BAUD_DIV 107  // 115200 baud at 100 MHz

volatile char rx_buffer[256];
volatile uint8_t rx_head = 0;
volatile uint8_t rx_tail = 0;

void external_irq_handler(void) {
    uint32_t src = rv_plic_claim();

    if (src == RV_PLIC_SRC_UART) {
        uint32_t status = rv_uart_irq_status();

        if (status & RV_UART_IE_RX_READY) {
            // Read all available characters
            while (rv_uart_rx_ready()) {
                rx_buffer[rx_head++] = rv_uart_getc();
            }
        }
    }

    rv_plic_complete(src);
}

int main(void) {
    // Initialize UART
    rv_uart_init(UART_BAUD_DIV);
    rv_uart_puts("UART Echo Demo\n");

    // Configure PLIC and enable interrupts
    rv_plic_init_source(RV_PLIC_SRC_UART, 5);
    rv_uart_irq_enable(RV_UART_IE_RX_READY);
    rv_irq_enable();

    // Main loop: echo received characters
    while (1) {
        if (rx_head != rx_tail) {
            char c = rx_buffer[rx_tail++];
            rv_uart_putc(c);
        }
    }
```

```
    return 0;
}
```

13.2. Example 2: SPI Flash Read

```
#include "rv_platform.h"
#include "rv_spi.h"

#define SPI_CLK_DIV 49  // 1 MHz at 100 MHz system clock
#define FLASH_CMD_READ 0x03

void spi_flash_read(uint32_t addr, uint8_t *buf, uint32_t len) {
    uint8_t cmd[4];
    cmd[0] = FLASH_CMD_READ;
    cmd[1] = (addr >> 16) & 0xFF;
    cmd[2] = (addr >> 8) & 0xFF;
    cmd[3] = addr & 0xFF;

    rv_spi_cs_select(0);
    rv_spi_transfer_buf(cmd, NULL, 4);
    rv_spi_read_buf(buf, len);
    rv_spi_cs_deselect();
}

int main(void) {
    uint8_t data[256];

    // Initialize SPI
    rv_spi_init(SPI_CLK_DIV, RV_SPI_MODE0);

    // Read 256 bytes from address 0x00010000
    spi_flash_read(0x00010000, data, 256);

    // Process data...

    rv_magic_exit(0);
    return 0;
}
```

13.3. Example 3: I2C EEPROM Access

```
#include "rv_platform.h"
#include "rv_i2c.h"

#define I2C_CLK_DIV 249  // 100 kHz at 100 MHz
#define EEPROM_ADDR 0x50
```

```c
int eeprom_write(uint16_t mem_addr, const uint8_t *data, uint8_t len) {
    uint8_t buf[66];  // addr(2) + data(64 max)

    if (len > 64) return -1;

    buf[0] = (mem_addr >> 8) & 0xFF;
    buf[1] = mem_addr & 0xFF;
    for (uint8_t i = 0; i < len; i++) {
        buf[i + 2] = data[i];
    }

    return rv_i2c_master_write(EEPROM_ADDR, buf, len + 2);
}

int eeprom_read(uint16_t mem_addr, uint8_t *data, uint8_t len) {
    uint8_t addr_buf[2];
    addr_buf[0] = (mem_addr >> 8) & 0xFF;
    addr_buf[1] = mem_addr & 0xFF;

    // Write address
    rv_i2c_start();
    if (rv_i2c_write_byte((EEPROM_ADDR << 1) | 0) < 0) {
        rv_i2c_stop();
        return -1;
    }
    rv_i2c_write_byte(addr_buf[0]);
    rv_i2c_write_byte(addr_buf[1]);

    // Read data
    rv_i2c_start();  // Repeated START
    if (rv_i2c_write_byte((EEPROM_ADDR << 1) | 1) < 0) {
        rv_i2c_stop();
        return -1;
    }
    for (uint8_t i = 0; i < len; i++) {
        data[i] = rv_i2c_read_byte(i < len - 1);
    }
    rv_i2c_stop();

    return 0;
}

int main(void) {
    uint8_t write_data[] = {0xAA, 0xBB, 0xCC, 0xDD};
    uint8_t read_data[4];

    rv_i2c_init(I2C_CLK_DIV);

    // Write to EEPROM
    if (eeprom_write(0x0100, write_data, 4) < 0) {
```

```
        rv_magic_exit(1);
    }

    // Small delay for write cycle
    for (volatile int i = 0; i < 10000; i++);

    // Read back
    if (eeprom_read(0x0100, read_data, 4) < 0) {
        rv_magic_exit(2);
    }

    // Verify
    for (int i = 0; i < 4; i++) {
        if (read_data[i] != write_data[i]) {
            rv_magic_exit(3);
        }
    }

    rv_magic_exit(0);
    return 0;
}
```

13.4. Example 4: DMA Memory Copy with Performance Monitoring

```
#include "rv_platform.h"

#define DMA_CHANNEL 0
#define DATA_SIZE 4096

__attribute__((aligned(4))) uint8_t src_buffer[DATA_SIZE];
__attribute__((aligned(4))) uint8_t dst_buffer[DATA_SIZE];

void dma_memcpy(void *dst, const void *src, uint32_t len) {
    // Configure DMA channel 0 for 1D transfer
    RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_SRC_OFF) = (uint32_t)src;
    RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_DST_OFF) = (uint32_t)dst;
    RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_XFER_OFF) = len;

    // Reset and enable performance counters
    RV_DMA_GLB_REG(RV_DMA_PERF_CTRL_OFF) = 0x3;

    // Start transfer
    RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_CTRL_OFF) =
        RV_DMA_CTRL_EN | RV_DMA_CTRL_MODE_1D |
        RV_DMA_CTRL_SRC_INC | RV_DMA_CTRL_DST_INC | RV_DMA_CTRL_START;

    // Wait for completion
```

```
    while (RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_STAT_OFF) & RV_DMA_STAT_BUSY);

    // Check for errors
    uint32_t stat = RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_STAT_OFF);
    if (stat & RV_DMA_STAT_ERR) {
        rv_magic_exit(1);
    }

    // Clear done flag
    RV_DMA_CH_REG(DMA_CHANNEL, RV_DMA_CH_STAT_OFF) = RV_DMA_STAT_DONE;
}

int main(void) {
    // Initialize source buffer
    for (int i = 0; i < DATA_SIZE; i++) {
        src_buffer[i] = i & 0xFF;
    }

    // Perform DMA copy
    dma_memcpy(dst_buffer, src_buffer, DATA_SIZE);

    // Read performance counters
    uint32_t cycles = RV_DMA_GLB_REG(RV_DMA_PERF_CYCLES_OFF);
    uint32_t rd_bytes = RV_DMA_GLB_REG(RV_DMA_PERF_RD_BYTES_OFF);
    uint32_t wr_bytes = RV_DMA_GLB_REG(RV_DMA_PERF_WR_BYTES_OFF);

    // Verify data
    for (int i = 0; i < DATA_SIZE; i++) {
        if (dst_buffer[i] != src_buffer[i]) {
            rv_magic_exit(2);
        }
    }

    rv_magic_exit(0);
    return 0;
}
```

Chapter 14. Build System Integration

14.1. Makefile Configuration

The SDK headers are located in `sw/include/`. Add this directory to your include path:

```
CFLAGS = -march=rv32ima_zicsr -mabi=ilp32 -O2 -g
CFLAGS += -I$(PROJECT_ROOT)/sw/include
```

14.2. Linker Script

The standard linker script (`sw/common/link.ld`) defines:

- `.text` section at `0x80000000` (RAM)
- `.data` and `.bss` sections following `.text`
- Stack pointer initialized to end of RAM
- Vector table at the start of `.text`

14.3. Startup Code

The startup code (`sw/common/start.S`) provides:

- Reset vector and exception vector table
- Stack pointer initialization
- BSS clearing
- Data section initialization (if loaded from ROM)
- Jump to `main()`

14.4. Common Runtime

Common runtime files in `sw/common/`:

- `trap.c`: Default exception handler
- `syscall.c`: Minimal newlib syscall stubs
- `printf.c`, `puts.c`, `putc.c`: Basic I/O functions

Chapter 15. Appendix A: Register Quick Reference

15.1. UART Registers

Register	Offset	Description
DATA	0x00	TX/RX data
STATUS	0x04	TX_BUSY(0), RX_READY(2)
IE	0x08	Interrupt enable
IS	0x0C	Interrupt status (W1C)
LEVEL	0x10	Baud rate divisor
CTRL	0x14	LOOPBACK(0)

15.2. SPI Registers

Register	Offset	Description
CTRL	0x00	EN(0), CPOL(1), CPHA(2), CS[7:4]
DIV	0x04	Clock divisor
TX	0x08	TX FIFO push (WO)
RX	0x0C	RX FIFO pop (RO)
STATUS	0x10	BUSY(0), TX_READY(1), RX_VALID(2)
IE	0x14	Interrupt enable
IS	0x18	Interrupt status (W1C)

15.3. I2C Registers

Register	Offset	Description
CTRL	0x00	EN(0), START(1), STOP(2), READ(3), NACK(4)
DIV	0x04	Clock divisor
TX	0x08	Transmit data (WO)
RX	0x0C	Receive data (RO)
STATUS	0x10	BUSY(0), TX_READY(1), RX_VALID(2), ACK_RECV(3)
IE	0x14	Interrupt enable
IS	0x18	Interrupt status (W1C)

15.4. DMA Channel Registers

Register	Offset	Description
CTRL	0x00	Control and mode selection
STAT	0x04	BUSY(0), DONE(1), ERR(2)
SRC	0x08	Source address
DST	0x0C	Destination address
XFER	0x10	Transfer size (bytes)
SSTRIDE	0x14	Source stride (2D/3D)
DSTRIDE	0x18	Destination stride (2D/3D)
ROWCNT	0x1C	Row count (2D/3D)

Chapter 16. Appendix B: Interrupt Vector Table

The default trap handler (`sw/common/trap.c`) dispatches interrupts and exceptions:

Code	Type	Handler
3	Machine Software Interrupt	Software interrupt handler
7	Machine Timer Interrupt	Timer interrupt handler
11	Machine External Interrupt	PLIC claim/complete sequence
0	Instruction Misaligned	Exception handler
2	Illegal Instruction	Exception handler
11	ECALL	System call handler

Chapter 17. Appendix C: Performance Considerations

17.1. DMA vs CPU Copy

For bulk data transfers > 256 bytes, DMA provides significant performance benefits:

- **CPU Copy**: ~4-8 cycles per word (load + store + loop overhead)
- **DMA**: 1-2 cycles per word (overlapped read/write, no CPU involvement)

17.2. Cache Coherency

The RV32 core includes an instruction cache but no data cache. When using DMA:

- No explicit cache maintenance required for data transfers
- Use `fence` instruction after MMIO writes to ensure ordering

17.3. Interrupt Latency

Typical interrupt latency (cycles from assertion to handler entry):

- Minimum: ~15 cycles (no contention)
- Average: ~25 cycles (with PLIC arbitration)
- Maximum: ~100 cycles (worst-case pipeline state)

Chapter 18. Appendix D: Debugging Tips

18.1. Enable Instruction Trace

```
make rtl-mytest TRACE=1
```

Generates `rtl_trace.txt` with PC, instruction, and register values.

18.2. Generate Waveforms

```
make rtl-mytest WAVE=fst
```

Creates `wave.fst` for viewing in GTKWave.

18.3. Common Issues

No output from UART

Check baud rate divisor calculation

DMA transfer fails

Verify source and destination addresses are within valid memory ranges

Interrupts not firing

Ensure global interrupts enabled (`rv_irq_enable()`)

I2C NACK errors

Check device address and pull-up resistors on SCL/SDA

Document Information

Version: 1.0

Last Updated: 2026-02-25

Copyright: © 2026 RV32 SoC Project

License: This documentation is provided under the MIT License.

For the latest version of this document and SDK updates, visit the project repository.